BỘ XÂY DỰNG

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI TPHCM

VIỆN ĐÀO TẠO CHẤT LƯỢNG CAO

NETWORK PROGRAMMING REPORT

Project title:

DEVELOPMENT OF AN ONLINE MEETING APPLICATION

Lecturer:	**Mai Ngọc Châu**
Students: **Huỳnh Dương Minh Triết** **Cao Minh Trí** **Đặng Thị Bảo Trân** **Phan Nhật Trường** **Hoàng Nguyễn Xuân Tuyền** **Nguyễn Hoàng Phát**	**ID:** **052306002763** **075206024645** **080306010991** **080206005713** **075306000328** **080206004349**
Class	**012012301305**

2026

Table of Contents

CHAPTER 1. INTRODUCTION 3
1.1. Motivation 3
1.2. Objectives 3
CHAPTER 2. SYSTEM OVERVIEW 4
2.1. System Overview Description 4
2.2. Roles in the System 4
2.3. Basic Workflow 4
CHAPTER 3. SYSTEM ARCHITECTURE 5
3.1. Overall Architecture 5
3.2. System Workflow 5
CHAPTER 4. SYSTEM DESIGN 7
4.1. Model Design 7
4.2. Room Management Design 7
4.3. Network Layer Design 7
4.4. Server Design 8
4.5. Client Design 8
4.6. Host Design 8
4.7. Features Design 9
CHAPTER 5. IMPLEMENTATION 10
5.1 Programming Languages and Technologies Used 10
5.2 Server Implementation 10
5.3 TCP Socket Communication 10
5.4 Multi-threading 11
5.5 Client Handling 11
CHAPTER 6. KEY FEATURES 13
CHAPTER 7. TESTING & EVALUATION 14
7.1. Test Cases 14
7.2. Evaluation 14
CHAPTER 8. LIMITATIONS OF THE SYSTEM 15
CHAPTER 9. CONCLUSION 16

CHAPTER 1. INTRODUCTION

1.1. Motivation

The rapid development of information technology has significantly transformed the way people communicate, study, and work. In particular, online meeting solutions have become increasingly popular, enabling users to connect and exchange information without geographical limitations. Platforms such as Google Meet, Zoom, Microsoft Teams, and Cisco Webex play a crucial role in supporting remote learning and working environments.

Motivated by these practical demands, developing an online meeting application is a suitable approach to gaining a deeper understanding of computer networking principles. Through this project, it is possible to explore how to establish connections among multiple users, manage data transmission within a system, and handle multiple clients operating concurrently in a shared environment.

This project is implemented using the Transmission Control Protocol (TCP) in combination with the client–server architecture, aiming to build a basic online meeting system that can serve as a foundation for future enhancements.

1.2. Objectives

The primary objective of this project is to design and develop a basic online meeting system based on the client–server model, allowing multiple users to participate and communicate within a shared virtual space.

Specifically, the project aims to:

Develop mechanisms for creating and joining meeting rooms

Establish communication between clients using TCP socket connections

Support real-time message exchange among participants

Manage the list of participants and maintain the state of meeting rooms

Ensure the system can handle multiple concurrent connections and operate reliably

Through the implementation process, this project helps reinforce knowledge of network programming while also improving skills in software design and system organization.

CHAPTER 2. SYSTEM OVERVIEW

2.1. System Overview Description

The online classroom system is developed based on the client–server model using C#, Socket, and TCP protocol to ensure reliable and real-time communication.

The system consists of main modules: Server (ChatServer, ClientHandler), Models (User, Message), Managers (RoomManager), Network (SocketManager, MessageParser), and additional Features such as PrivateChat, FileTransfer, and EmojiHandler.

The system allows multiple clients to connect simultaneously, create or join rooms, and communicate in real time through chat. TCP ensures data is transmitted reliably, in order, and without loss.

2.2. Roles in the System

Host: Creates and manages the room, including participant list and activities.

Client: Connects to the server, joins rooms, sends/receives messages, and uses features such as private chat and file transfer.

Server: Acts as the central component, handling connections, processing requests, managing rooms, and broadcasting messages to clients. It supports multiple clients simultaneously and maintains system synchronization.

2.3. Basic Workflow

Workflow:

Host → Server → RoomManager → Client
↓
Broadcast

The system operates with a centralized processing model at the Server. When a client sends a request, the Server receives and processes it, then forwards it to RoomManager for handling room-related logic. After processing, the Server sends data to clients and performs broadcast to all users in the same room. This ensures real-time communication and synchronization among all users in the system.

CHAPTER 3. SYSTEM ARCHITECTURE

3.1. Overall Architecture

The system is designed based on the Client–Server model using the TCP protocol, where the server acts as the central unit responsible for processing and managing data, while clients serve as the interface for user interaction. In this model, clients do not communicate directly with each other but instead send all requests through the server, which ensures better data control, reduces conflicts, and supports multiple users simultaneously. This architecture is suitable for real-world chat applications because it allows flexible scalability, meaning the system can handle an increasing number of users without requiring major structural changes.

The system is divided into several main components. The Models component defines the core data structures, such as the User class, which stores user information including name, joined room, and permissions, and the Message class, which contains message details such as sender, content, and timestamp. The Managers component, specifically RoomManager, is responsible for managing chat rooms by handling tasks such as creating rooms, allowing users to join or leave, maintaining user lists, and broadcasting messages to all members within a room. The Network component serves as the communication layer, where SocketManager manages connections and data transmission through TCP, and MessageParser converts data between object format and string format to support network communication. On the server side, ChatServer is responsible for starting the server, listening for incoming connections, and accepting clients, while ClientHandler processes each client individually, allowing the system to handle multiple users at the same time. On the client side, ChatClient manages communication with the server, ClientUIConsole provides the user interface, and the Host component represents a client with additional permissions such as creating rooms and managing participants. Overall, the architecture is modular, with each component having a clearly defined role, which makes the system easier to maintain and extend.

3.2. System Workflow

The system workflow describes how a message is transmitted from one user to others within the same chat room. First, the user enters a message through the ClientUIConsole interface, and this input is passed to ChatClient, where it is packaged into a Message object containing the sender, content, and timestamp. The message is then sent to the server via TCP using SocketManager, with the data converted into string format for transmission. Upon receiving the message, the server forwards it to the corresponding ClientHandler, which processes the data and sends it to RoomManager to determine the appropriate chat room. RoomManager then retrieves the list of users in that room and broadcasts the message to all members, ensuring that every participant receives the same content. On the client side, each application continuously listens for incoming data, and when a message is received, ChatClient

processes it and displays it on the ClientUIConsole interface. This process occurs continuously and in near real time, and thanks to TCP, messages are delivered reliably, in the correct order, and without loss. In summary, the workflow can be described as a simple process where the client sends a message to the server, the server processes it, and then distributes it to other clients in the same room, ensuring synchronization and stability across the system.

CHAPTER 4. SYSTEM DESIGN

4.1. Model Design

The system is built upon two fundamental models: User and Message, which define the core data structures for communication and interaction.

The User class represents a participant in the system. It stores essential information such as username, current room ID, host role, and TCP connection. This design allows the server to efficiently manage users, track their activities within rooms, and handle communication through network sockets. By embedding the TCP connection directly inside the User object, the system can send messages without additional lookup mechanisms, thereby simplifying communication and improving performance.

The Message class defines the structure of all communication data exchanged between users. Each message includes the sender, content, timestamp, and a specific type such as JOIN, CHAT, LEAVE, SYSTEM, or PRIVATE. This classification standardizes message handling and simplifies server-side processing. Additionally, consistent formatting ensures reliability and makes it easier to extend the system in the future, for example by adding new message types like file sharing or notifications.

4.2. Room Management Design

Room management is handled by the RoomManager, which is responsible for maintaining rooms and coordinating user interactions. It uses a dictionary structure that maps room IDs to lists of users, enabling fast access and efficient handling of multiple rooms simultaneously.

The RoomManager provides core functionalities such as creating rooms with unique identifiers, allowing users to join or leave rooms, and broadcasting messages to all participants. When a user leaves a room and no participants remain, the system automatically deletes the room to optimize resource usage.

In addition, the system supports administrative operations such as removing users (KickUser) and closing rooms (CloseRoom). These actions ensure proper cleanup of connections and notify other users, helping maintain consistency and control within the system. Overall, this component plays a central role in managing room lifecycles and synchronizing data among connected clients.

4.3. Network Layer Design

The network layer ensures reliable communication between client and server through two main components: SocketManager and MessageParser.

The SocketManager handles TCP connections by establishing links, sending data, and receiving messages via network streams. It acts as a bridge between the client and

server, ensuring stable and continuous communication. By abstracting low-level socket operations, it simplifies higher-level logic and promotes code reuse.

The MessageParser is responsible for converting structured messages into string format for transmission and parsing incoming data back into structured objects. This ensures consistency in communication between client and server, reduces errors, and makes the protocol easier to maintain and extend.

4.4. Server Design

The server architecture consists of ChatServer and ClientHandler, which together manage connections and communication.

The ChatServer initializes the system, listens for incoming client connections, and manages multiple users concurrently. It adopts a multi-threaded approach, where each client connection is handled in a separate thread. This allows the server to process multiple requests simultaneously without blocking, which is essential for real-time communication systems.

The ClientHandler is responsible for managing communication with individual clients. It processes requests such as creating rooms, joining rooms, sending messages, and private communication. It interacts with the RoomManager to perform room-related operations and ensures messages are delivered to appropriate users. Additionally, it manages the lifecycle of each client connection, from connection establishment to disconnection, ensuring system stability.

4.5. Client Design

The client side is divided into two main components: ChatClient and ClientUIConsole, ensuring a clear separation of concerns.

The ChatClient manages the connection to the server and handles sending and receiving messages. It acts as an intermediary between the server and the user interface. By separating communication logic from the UI, the system becomes more modular, easier to maintain, and more flexible for future enhancements.

The ClientUIConsole provides a console-based interface for user interaction. It captures user input and displays messages received from the server in real time. This separation allows the interface to be upgraded (e.g., to a graphical UI) without affecting the underlying communication logic.

4.6. Host Design

The Host plays a crucial role in managing meeting rooms. The host is responsible for creating rooms, maintaining room activity, and controlling participants.

Key functionalities include closing rooms, removing users, and viewing participant lists. These capabilities ensure that sessions remain organized and controlled. By distinguishing the host role from regular users, the system supports better management and reflects real-world meeting scenarios.

4.7. Features Design

The system includes several additional features to enhance functionality and user experience.

The FileTransfer feature allows users to share files by converting them into byte arrays and transmitting them in smaller chunks over TCP. This method prevents network congestion and ensures stable data transmission, especially for large files.

The PrivateChat feature enables direct communication between two users. Instead of broadcasting messages to all participants, the system sends messages only to the intended recipient, ensuring privacy and reducing unnecessary network traffic.

The EmojiHandler improves user experience by replacing predefined text patterns with corresponding emoji characters. This enhances readability and makes communication more expressive, particularly in a console-based environment.

CHAPTER 5. IMPLEMENTATION

5.1 Programming Languages and Technologies Used

The system is developed using the C# programming language on the .NET platform. Choosing C# allows leveraging powerful libraries for network programming such as System.Net and System.Net.Sockets, while also providing efficient multithreading support through the System.Threading namespace. In addition, the system uses the TCP protocol to ensure that data transmission between components is stable, accurate, and in the correct order.

5.2 Server Implementation

The server is the central component of the system, responsible for receiving and handling connections from clients. The server is implemented through the ChatServer class, using TcpListener to listen for incoming client connections. When the system starts, a separate thread is created to continuously accept new connections.

Each time a client successfully connects, the system creates a corresponding ClientHandler object and executes it on a separate thread, ensuring that multiple clients can be handled simultaneously without causing system congestion.

```
var client = server.AcceptTcpClient();
Console.WriteLine("Client connected");

var handler = new ClientHandler(client, roomManager);
new Thread(handler.HandleClient) { IsBackground = true }.Start();
```

5.3 TCP Socket Communication

The system uses TCP sockets to establish connections between the Server and Clients. On the server side, TcpListener is used to listen for incoming connections, while on the client side, TcpClient is used to initiate the connection. Data is transmitted in string format through NetworkStream and then processed using the StreamReader and StreamWriter classes.

Using TCP provides several advantages such as ensuring no data loss, maintaining correct transmission order, and offering high reliability, making it highly suitable for real-time communication systems.

5.4 Multi-threading

To handle multiple users accessing the system simultaneously, a multithreading mechanism is applied, where each client is processed on a separate thread. This allows the server to accept new connections while simultaneously handling data from existing clients without interruption.

The use of multithreading improves system performance and scalability. However, it also requires proper resource management to avoid issues such as data conflicts or excessive memory usage.

5.5 Client Handling

All server-side processing logic is implemented in the ClientHandler class. When a client connects, the system first receives the initial username, then continuously listens for requests sent from the client via StreamReader.

The system supports various types of requests such as creating a room (CREATE), joining a room (JOIN), sending messages, sending private messages (PRIVATE), kicking users (KICK), and closing a room (CLOSE). Each type of request is identified by a specific prefix in the data string and handled accordingly.

When a client sends a request to join a room, the system parses the received string to extract the room ID and performs validation checks:

```
if (msg.StartsWith("JOIN|"))
{
    string roomId = msg.Split('|')[1];

    bool ok = roomManager.JoinRoom(roomId, user);
```

If the room exists, the user will be added to the member list, and the system will send updated information to all clients in the room. Otherwise, the server will return an error message.

Room management is handled through the RoomManager class. This class uses a data structure of type Dictionary<string, List<User>> to store the list of rooms and

their corresponding users. Functions such as creating rooms, adding users, removing users, broadcasting messages, and updating room information are all processed here.

In particular, when a user sends a message, the system broadcasts it to all clients within the same room. Before sending, the message content is processed through the EmojiHandler to convert special characters into emojis, enhancing the system's visual experience.

On the client side, the system is implemented using the ChatClient class along with console-based interface classes. The client uses TcpClient to connect to the server and establishes a communication channel via NetworkStream. After a successful connection, the client sends its username to the server and begins data exchange.

A separate thread is created to continuously receive data from the server:

```
new Thread(Receive).Start();
```

The Receive() method runs an infinite loop to ensure that all information from the server is received and displayed immediately. This enables the system to achieve real-time communication.

CHAPTER 6. KEY FEATURES

The system provides a comprehensive online communication solution with the following features:

Group Chat: The core feature allowing all room members to receive messages from one another in real-time.

Quick Join (Join by link/ID): Users can join discussion rooms via a unique RoomID, making the connection process straightforward and fast.

Host Control: The room creator (Host) holds supreme administrative privileges, including viewing the online member list, kicking users, or closing the entire chat room.

Private Messaging: Supports confidential conversations between two individuals within the same room without appearing on the public chat stream.

File Sharing: Facilitates sending and receiving document files directly via TCP connections with high stability.

Emoji Support: Livens up conversations with a rich Unicode character system, rendered smoothly on the Console UI.

CHAPTER 7. TESTING & EVALUATION

7.1. Test Cases

The system underwent rigorous testing in Local Area Network (LAN) and Localhost environments based on the following criteria:

Concurrency & Load Testing: Simultaneously connecting 4-10 Clients to a single chat room. Continuously typing and sending messages to evaluate the Server's asynchronous processing capabilities. The Server employs a list-locking mechanism (.ToList()) to prevent Data Race Conditions during iterations.

Stability Testing: Simulating scenarios where a Client abruptly closes the Console window or loses network connection. Ensuring that the Server detects disconnected sockets via Try-Catch blocks, immediately releases thread resources, and notifies the remaining Clients without causing a Server crash.

Data Integrity Testing: Sending long text strings, files ranging from 1-5MB, and strings containing Emoji codes to verify that there is no packet loss or data corruption over the TCP protocol.

7.2. Evaluation

Performance: Operates with ultra-low latency. The Multi-threading architecture separates the background receiving thread from the main input thread, preventing UI freezing and text overlapping.

Reliability: Fault-tolerant and highly efficient connection management.

Scalability: The clear Layered Architecture adheres to encapsulation principles, facilitating future upgrades to WinForms or WPF.

Limitations: Lacks database integration for chat history. Public Internet deployment requires Port Forwarding configurations.

CHAPTER 8. LIMITATIONS OF THE SYSTEM

The online meeting system has completed the basic functions, but there are still some limitations that need improvement for better real-world usage.

The user interface is currently simple and mainly used for demonstration and testing purposes. The layout is not fully optimized, so the user experience is not very smooth. When there is a large amount of information displayed or when users interact frequently, the interface may become less clear and harder to use. A more modern and user-friendly design is needed to improve usability.

The system currently works mainly in a local network environment. It has not been deployed on the Internet, so users cannot connect from different locations. This limits its real-world usage, especially in situations where people need to join meetings remotely from different places.

The system has only been tested with a small number of users. When many users join at the same time, the system may face problems such as slower performance, delays, or reduced stability. This is because the system has not been fully optimized for handling a large number of connections.

Security features are not included in this version. Functions such as user authentication, data encryption, and access control are still missing. This makes the system less safe for real use, especially when sensitive information is shared.

In addition, resource management and performance optimization are still basic. The system does not use advanced techniques to handle large workloads or improve efficiency. This means it may not perform well under heavy usage and needs further improvement in the future.

Overall, while the system works well for basic purposes, it still needs several improvements to become more stable, secure, and suitable for real-world applications.

CHAPTER 9. CONCLUSION

This project has successfully built a basic online meeting system based on the Client–Server model using TCP. The system allows multiple users to join the same room and communicate in real time. It also supports important functions such as creating a room, joining a room, and managing users within the meeting.

Through this project, the working process of a network system becomes clearer. It includes creating connections between client and server, sending and receiving data, and managing multiple users at the same time. These processes help improve understanding of how real-time communication systems work in practice.

The system is designed with separate components such as server, host, and client. This structure makes the system easier to understand, maintain, and develop further. The use of both Windows Forms and Console also shows flexibility in implementation, allowing the system to be adapted to different environments.

In the future, the system can be improved in many ways to increase its usefulness. It can add real-time voice and video features to make communication more natural and interactive. These features will help users feel more connected, similar to real meeting applications.

The system can also support screen sharing so users can present documents or show their screen during meetings. This is very helpful for learning, teamwork, and presentations.

Other improvements include developing a better user interface to make the system easier to use, adding stronger security features to protect user data, and improving performance so the system can handle more users at the same time.

With these improvements, the system can become more complete and closer to real-world online meeting applications.